

K29: ΣΧΕΔΙΑΣΜΟΣ ΚΑΙ ΧΡΗΣΗ ΒΑΣΕΩΝ ΔΕΔΟΜΕΝΩΝ

ΕΡΓΑΣΙΑ 3

Θεοδώρα Κόσσυφα - sdi2300081

Η παρούσα εργασία αφορά τον σχεδιασμό και την υλοποίηση του k29photo, μιας διαδικτυακής πλατφόρμας κοινοποίησης φωτογραφιών εμπνευσμένης από το Flickr. Η εφαρμογή υλοποιήθηκε με χρήση PostgreSQL για τη διαχείριση της βάσης δεδομένων και Flask (Python 3) για το backend της διαδικτυακής εφαρμογής.

Φάση I - Σχεδιασμός Βάσης Δεδομένων

E/R Διάγραμμα: Το E/R διάγραμμα του συστήματος περιλαμβάνει τις εξής οντότητες και συσχετίσεις:

Οντότητες:

- User: Χρήστης της πλατφόρμας
- Album: Άλμπουμ φωτογραφιών
- Photo: Φωτογραφία που ανεβαίνει στην πλατφόρμα
- Tag: Ετικέτα κατηγοριοποίησης φωτογραφιών
- Comment: Σχόλιο σε φωτογραφία

Συσχετίσεις:

- User *owns* Album (1:N)
- Album *contains* Photo (1:N)
- Photo *tagged_with* Tag (M:N)
- User *comments_on* Photo (M:N, μέσω Comments)\
- User *likes* Photo (M:N)
- User *friends_with* User (M:N, αυτο-αναφορική)

(Αρχείο er-diagram.mwb / er-diagram.png)

Σχεσιακό Σχήμα: Η μετάφραση του E/R διαγράμματος στο σχεσιακό μοντέλο παράγει τους εξής πίνακες:

users(user_id, first_name, last_name, email, dob, hometown, gender, password)

albums(album_id, name, owner_id → users, date_created)

photos(photo_id, caption, data, album_id → albums)

tags(tag_id, name)

photo_tags(photo_id → photos, tag_id → tags)

comments(comment_id, text, owner_id → users, photo_id → photos, post_date)

friends(user_id1 → users, user_id2 → users)

likes(user_id → users, photo_id → photos)

Κάθε M:N συσχέτιση (photo_tags, friends, likes) αναπαρίσταται με ξεχωριστό πίνακα. Οι 1:N συσχετίσεις (albums→users, photos→albums) αναπαρίστανται με foreign keys.

Constraints

CHECK Constraints:

Πίνακας	Constraint	Περιγραφή
users	gender IN ('M','F','O')	Έγκυρες τιμές φύλου
tags	name = LOWER(name)	Lowercase ετικέτες
tags	name NOT LIKE '% %'	Χωρίς κενά
friends	user_id1 < user_id2	Αποφυγή διπλότυπων

UNIQUE Constraints:

- users.email: Μοναδική διεύθυνση email
- tags.name: Μοναδικό όνομα ετικέτας

FOREIGN KEY Constraints: Όλες οι εξαρτώμενες εγγραφές διαγράφονται αυτόματα όταν διαγράφεται ο γονικός πίνακας.

Triggers

Πίνακας	Trigger	Περιγραφή
comments	trg_check_comment_owner	Χρήστης δεν σχολιάζει δική του φωτό (NULL = guest, επιτρέπεται)
likes	trg_check_like_owner	Χρήστης δεν κάνει like δική του φωτό
friends	trg_check_self_friend	Δεν επιτρέπεται self-friendship
friends	trg_check_duplicate_friend	Δεν επιτρέπεται διπλή φιλία
photos	trg_check_photo_album_owner	Έλεγχος ύπαρξης album
likes	trg_check_duplicate_like	Δεν επιτρέπεται διπλό like
tags	trg_normalize_tag_name	Auto-lowercase & validation tag

Φάση II - Υλοποίηση Εφαρμογής

Τεχνολογίες

- Backend: Python 3, Flask
- Βάση Δεδομένων: PostgreSQL (psycopg2)
- Frontend: HTML5, CSS3, Jinja2 templates
- Session Management: Flask sessions

Υλοποίηση Use Cases

Διαχείριση Χρηστών:

Εγγραφή Χρήστη: Η εγγραφή γίνεται μέσω φόρμας με τα πεδία: όνομα, επίθετο, email, κωδικό πρόσβασης, πόλη, ημερομηνία γέννησης (προαιρετικό) και φύλο. Σε περίπτωση duplicate email εμφανίζεται κατάλληλο μήνυμα σφάλματος μέσω του UNIQUE constraint.

Προσθήκη & Προβολή Φίλων: Η αναζήτηση χρηστών γίνεται με LIKE query σε όνομα, επίθετο και email. Η φιλία είναι αμφίδρομη δλδ όταν ο A προσθέτει τον B, και οι δύο

γίνονται φίλοι αυτόματα. Για αποφυγή διπλότυπων αποθηκεύεται πάντα user_id1 < user_id2.

Δραστηριότητα Χρηστών: Το contribution score υπολογίζεται δυναμικά:
score = COUNT(photos uploaded) + COUNT(comments on others' photos)
Οι 10 κορυφαίοι χρήστες εμφανίζονται στη σελίδα /top-users.

Διαχείριση Άλμπουμ και Φωτογραφιών:

Περιήγηση: Όλες οι φωτογραφίες και τα άλμπουμ είναι δημόσια και ορατά σε όλους τους επισκέπτες, εγγεγραμμένους και μη.

Δημιουργία & Διαγραφή: Μόνο εγγεγραμμένοι χρήστες μπορούν να δημιουργούν άλμπουμ και να ανεβάζουν φωτογραφίες. Η διαγραφή επιτρέπεται μόνο στον owner. Τα δυαδικά δεδομένα της εικόνας αποθηκεύονται ως BYTEA στην PostgreSQL.

Διαχείριση Ετικετών

Προβολή ανά Tag: Κάθε tag είναι clickable και εμφανίζει τις αντίστοιχες φωτογραφίες. Παρέχεται switch "Όλες οι Φωτογραφίες / Οι Δικές μου" για εναλλαγή προβολής.

Δημοφιλή Tags: Εμφανίζονται τα 20 πιο δημοφιλή tags ταξινομημένα κατά πλήθος φωτογραφιών.

Αναζήτηση: Υλοποιείται συζευκτική (AND) αναζήτηση ανά tags:

```
SELECT photo_id FROM photo_tags  
JOIN tags ON photo_tags.tag_id = tags.tag_id  
WHERE tags.name = ANY(tag_list)  
GROUP BY photo_id  
HAVING COUNT(DISTINCT tags.name) = len(tag_list)
```

Σχόλια:

Δημοσίευση: Τόσο οι εγγεγραμμένοι χρήστες όσο και οι επισκέπτες (guests) μπορούν να αφήνουν σχόλια. Τα guest σχόλια αποθηκεύονται με owner_id = NULL και εμφανίζονται ως "Επισκέπτης". Χρήστες δεν μπορούν να σχολιάζουν τις δικές τους φωτογραφίες (trigger 1).

Likes: Κάθε εγγεγραμμένος χρήστης μπορεί να κάνει like σε φωτογραφίες άλλων. Εμφανίζεται ο αριθμός likes και τα ονόματα των χρηστών που έκαναν like.

Αναζήτηση Σχολίων: Αναζήτηση με ακριβές κείμενο επιστρέφει τους χρήστες που έχουν γράψει το συγκεκριμένο σχόλιο, ταξινομημένους κατά πλήθος ταιριαστών σχολίων. Εμφανίζονται επίσης οι φωτογραφίες στις οποίες έχει γίνει το σχόλιο.

Προτάσεις φίλων:

Πρόταση Φίλου: Υλοποιείται ο αλγόριθμος friends-of-friends:

```
WITH my_friends AS (  
    SELECT CASE WHEN user_id1=X THEN user_id2 ELSE user_id1 END as fid  
    FROM friends WHERE user_id1=X OR user_id2=X  
)
```

```
SELECT u.user_id, COUNT(*) as common
FROM my_friends mf
JOIN friends f ON (f.user_id1=mf.fid OR f.user_id2=mf.fid)
JOIN users u ON (... = u.user_id)
WHERE u.user_id NOT IN (SELECT fid FROM my_friends)
GROUP BY u.user_id ORDER BY common DESC
```

You-May-Also-Like: Βρίσκονται οι 5 πιο συχνές ετικέτες του χρήστη και εκτελείται διαζευκτική αναζήτηση. Τα αποτελέσματα ταξινομούνται κατά πλήθος ταιριαστών tags (φθίνουσα) και κατά συνολικό πλήθος tags της φωτογραφίας (αύξουσα).

Παραδοχές

1. Αμφίδρομη Φιλία: Επιλέχθηκε αμφίδρομο μοντέλο φιλίας (όταν A προσθέτει B, η φιλία ισχύει και αντίστροφα). Αποθηκεύεται ένα ζεύγος με `user_id1 < user_id2` για αποφυγή διπλότυπων.
2. Guest Comments: Η εκφώνηση επιτρέπει σχόλια από επισκέπτες. Αυτό υλοποιήθηκε με nullable `owner_id` στον πίνακα `comments` και ενημέρωση του αντίστοιχου trigger.
3. Δημόσιες Φωτογραφίες: Όλες οι φωτογραφίες και τα άλμπουμ είναι δημόσια.
4. Αποθήκευση Εικόνων: Τα δυαδικά δεδομένα αποθηκεύονται ως `BYTEA` στην PostgreSQL.
5. Authentication: Η σύνδεση χρηστών υλοποιείται με απλή επαλήθευση email/password μέσω SQL query και Flask sessions, χωρίς χρήση επιπλέον βιβλιοθηκών (π.χ. flask_login).
6. Contribution Score: Μετράει φωτογραφίες που ανέβασε ο χρήστης συν σχόλια σε φωτογραφίες άλλων χρηστών. Το score ενημερώνεται δυναμικά, όταν διαγράφεται φωτογραφία ή σχόλιο, το score μειώνεται αυτόματα.

Παρατηρήσεις & Πιθανές Παραλείψεις

- Password Hashing: Οι κωδικοί αποθηκεύονται ως plaintext για λόγους απλότητας. Σε production περιβάλλον θα χρησιμοποιούνταν bcrypt ή παρόμοια βιβλιοθήκη.
- Pagination: Δεν υλοποιήθηκε σελιδοποίηση για μεγάλο αριθμό φωτογραφιών.
- Image Optimization: Δεν γίνεται resize/compression των εικόνων κατά το upload.

How to run the app

Ο κώδικας είναι διαθέσιμος στο GitHub: <https://github.com/kossyfa/k29photo>

1. Κλωνοποίηση Repository:

```
git clone https://github.com/kossyfa/k29photo.git
```

```
cd k29photo
```

2. Δημιουργία Virtual Environment

```
python3 -m venv myvirenv
```

```
source myvirenv/bin/activate
```

```
pip install -r requirements.txt
```

3. Δημιουργία Βάσης Δεδομένων

```
# Σύνδεση ως postgres admin
```

```
sudo -i -u postgres psql
```

```
# Μέσα στο psql:
```

```
CREATE USER k29 WITH PASSWORD '1234';
```

```
CREATE DATABASE k29photo WITH OWNER k29;
```

```
GRANT ALL PRIVILEGES ON DATABASE k29photo TO k29;
```

```
\q
```

4. Δημιουργία Πινάκων & Triggers

```
psql -U k29 -d k29photo -h localhost -a -f schema.sql
```

5. Φόρτωση Δεδομένων

```
# Επιλογή 1 (προτεινόμενη): Με πραγματικές εικόνες (εικόνες στο images/)
```

```
python3 load_data.py
```

```
# Επιλογή 2: Χωρίς εικόνες
```

```
psql -U k29 -d k29photo -h localhost -a -f sample_data.sql
```

6. Εκκίνηση Εφαρμογής

```
source myvirenv/bin/activate
```

```
python3 app.py
```

7. Άνοιξε στον Browser

```
http://127.0.0.1:5000
```